DRRNets: Dynamic Recurrent Routing via Low-Rank Regularization in Recurrent Neural Networks

Dongjing Shan, Yong Luo, *Member, IEEE*, Xiongwei Zhang, and Chao Zhang, *Member, IEEE*

***Abstract*— Recurrent neural networks (RNNs) continue to show outstanding performance in sequence learning tasks such as language modeling, but it remains difficult to train RNNs for long sequences. The main challenges lie in the complex dependencies, gradient vanishing or exploding, and low resource requirement in model deployment. In order to address these challenges, we propose dynamic recurrent routing neural networks (DRRNets), which can: 1) shorten the recurrent lengths by allocating recurrent routes dynamically for different dependencies and 2) reduce the number of parameters significantly by imposing low-rank constraints on the fully connected layers. A novel optimization algorithm via low-rank constraint and sparsity projection is developed to train the network. We verify the effectiveness of the proposed method by comparing it with multiple competitive approaches in several popular sequential learning tasks, such as language modeling and speaker recognition. The results in terms of different criteria demonstrate the superiority of our proposed method.**

***Index Terms*— Long-term memory, low rank, recurrent neural network (RNN), sparsity projection, temporal dependency, vanishing gradients.**

I. Introduction

RECURRENT neural networks (RNNs) have been widely used in sequential learning and have achieved remarkable performance in a variety of tasks, such as language modeling [1], retrospecting human dynamics [2], machine translation [3], image captioning [4], [5], and natural language processing [6]. Some representative works, including the long short-term memory [7], [8] (LSTM) and gated recurrent unit [9] (GRU), are proposed to alleviate the gradient vanishing issue by introducing the gated cells and employing information latching strategy; some others adopt additional modules to tackle the issue of long-term memorization, for instance, Dong and Xu [2] constructed a retrospection module as a temporary memory to retrospect previous information; Graves *et al.* [10] proposed to store long-term information in external memory.

The challenges of RNNs in model training and deployment mainly lie in three aspects.

1) The complex dependencies in extremely long sequence, where related elements may be interrupted by the irrelevant ones, which are nevertheless important and should be kept in other dependencies. Therefore, one forward pass route is not sufficient to deal with such case.
2) Capacity of long-term memorization. The gradients back-propagated through a long time span tend to vanish or explode due to the multiplicative accumulation, and then weaken the capacity of memorizing long-term information.
3) Low resource requirement in deployment of a trained model. In some special cases such as deployment in portable devices that have limited computational power and internal memory, resource requirements, such as memory consumption and computational cost, should be minimized while maintaining the same level of performance.

In order to address these challenges, recent attempts often utilize the multitimescale neural architectures or path shortening techniques. For example, the works, including phased LSTM [11], hierarchical multiscale RNNs [12], and dilated LSTM [13], learn to capture dependencies from different time scales and such strategy can shorten the gradient propagation path by reducing time resolution; the learning-based RNNs [14] can ignore irrelevant information when reading long texts, Skip RNNs [15] can learn to skip state updates at certain timestamps and shorten the effective recurrent length, and segmented-memory RNNs [16] can break memory into segments and shorten the path by imitating human memorization. However, these models do not explicitly exploit dependencies in jumping order or lack the ability to capture multidependencies simultaneously (e.g., the Skip RNNs only establish one recurrent path for each input sequence), and multilayers are often adopted, which would introduce large amounts of parameters and result in high computational burden.

To remedy these drawbacks, we propose dynamic recurrent routing neural architectures (DRRNets for short) to capture the complex dependencies with jumping order on timestamps. The DRRNets can construct information pass routes for different

Manuscript received 12 July 2020; revised 12 January 2021 and 16 June 2021; accepted 13 August 2021. Date of publication 30 August 2021; date of current version 5 April 2023. This work was supported by the National Natural Science Foundation of China under Grant 62071484. *(Corresponding author: Dongjing Shan.)*

Dongjing Shan is with the Laboratory of Intelligent Information Processing, Army Engineering University, Chongqing 400035, China (e-mail: shandongjing@pku.edu.cn).

Yong Luo is with the National Engineering Research Center for Multimedia Software, School of Computer Science, Institute of Artificial Intelligence, Wuhan University, Wuhan 430072, China, and also with the Hubei Key Laboratory of Multimedia and Network Communication Engineering, Wuhan University, Wuhan 430072, China (e-mail: luoyong@whu.edu.cn).

Xiongwei Zhang is with the Speech Processing Laboratory, Army Engineering University, Nanjing 210007, China (e-mail: xwzhang9898@163.com).

Chao Zhang is with the Key Laboratory of Machine Perception (MOE), Peking University, Beijing 100871, China (e-mail: chzhang@cis.pku.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TNNLS.2021.3105818.

Digital Object Identifier 10.1109/TNNLS.2021.3105818

2162-237X © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

dependencies, and the total number of routes and the state entries for each route are allocated dynamically via a low-rank pointer matrix. In each route, the state update on a group of entries is performed according to the binary pointers in the matrix, and then, useful tokens (elements in the input sequence) are chosen for the corresponding dependency and information can be conveyed along a shortened path. We also dig more deeply into the low-rank property in the LSTM gates' fully connected layers, which are associated with multiroute state information. Low-rank constraint is imposed on the weights of corresponding hidden layers, and the rank is approximately the same as that of the pointer matrix. Similar to the convolution operation, low rankness can also reduce the number of parameters while maintaining the capacity. Two low-rank regularizations are introduced in the loss function and optimized in a successive manner.

The main differences of our DRRNets from the previous works are as follows.

1) Our model can establish recurrent routes inside the cell states and allocate desirable entries, and each route can skip state updates at certain timestamps according to the related dependency, whereas other models [11]–[13] usually construct routes from predetermined timescales by multilayers. This often cannot capture the dependencies accurately and may lead to a large number of parameters.
2) Our model is the first work that exploits low-rank property from the view of input structure, and both of the decoupling and low-rank properties are imposed on the fully connected layers. Existing works [17], [18] usually impose a convolutional structure on these layers, which may only work well for certain type of data such as images, or merely utilize decoupling property for reducing the number of parameters [19], [20].

We also design a novel optimization algorithm to train the network. The algorithm can effectively exploit the low-rank property of the routing pointer matrix and enforce low-rankness for the network weight parameters via ℓ_0 projection. We compare our model with many competitive counterparts in three popular applications, including sequential classification, language modeling, and speaker recognition. The experimental results demonstrate the superiority of our proposed method.

II. Neural Architecture

A. Dynamic Recurrent Routing Method

Given a sequence of tokens $X = (\mathbf{x}_1, \ldots, \mathbf{x}_T), \mathbf{x}_i \in \mathbb{R}^{n_1}$, where n_1 indicates the input size, there may exist several dependencies which begin/end at different timestamps and have skipping connections inside their scopes. Our goal is to establish dynamic routes via a pointer matrix and then capture the dependencies accurately. The architecture can be built on a variety of recurrent networks, including RNN, LSTM, and GRU, which generates memory state $\mathbf{s}_t \in \mathbb{R}^{n_2}$ recurrently by using a general state transition model S

$$\mathbf{s}_t = S(\mathbf{s}_{t-1}, x_t) \tag{1}$$

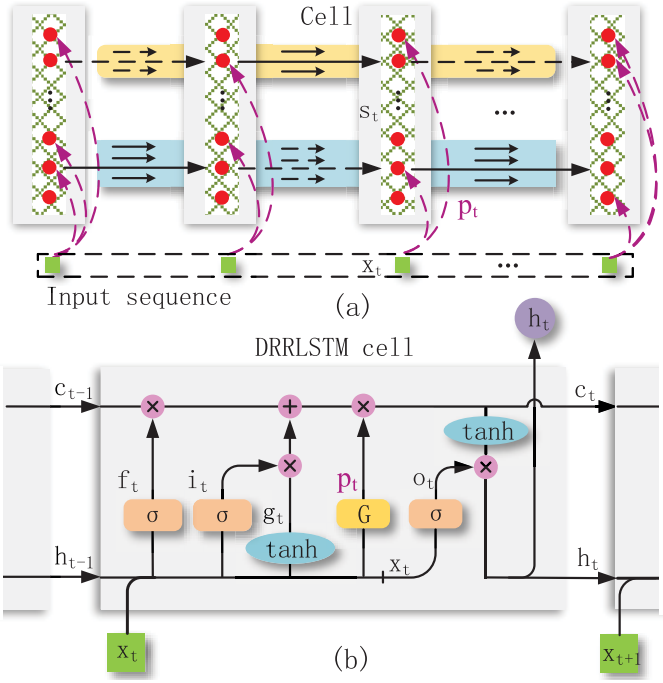


Fig. 1. (a) Forward pass of information among the cell states. The gray boxes represent recurrent cells and the inner rectangles represent cell states with numerous entries, and the cells are connected by solid or dotted arrows, which indicates that the state entries are updated with input information or copied from the previous time stamp. The color bands indicate different routes for state transmission, each with different number of entries allocated. (b) Cell structure of DRRLSTM where c_t acts as the state vector (i.e., s_t in the general transition model).

then, a binary pointer vector $\mathbf{p}_t \in \mathbb{R}^{n_2}$ is used to control updating of the entries in the cell state $\mathbf{s}_t$, by this way, the entries in the same row over the whole time line will form a recurrent path with width of one entry, and the paths may have skipping connections at different timestamps from each other. We need to cluster these paths into groups, and then, each group forms a route that aims to track information corresponding to one kind of dependency. Thus, the proposed dynamic recurrent routing means: 1) establishing routes with different recurrent lengths by using skip connections and 2) allocating different neuron resources for each route. To achieve this goal, we propose to constrain the pointer matrix $P_m = \{\mathbf{p}_1, \ldots, \mathbf{p}_T\} \in \mathbb{R}^{n_2 \times T}$ to be of low rank during training, where n_2 indicates the size of hidden layer (the same as the size of memory state) and T is the sequence length. The process of forming routes is shown in Fig. 1(a), and each linear correlation group in the matrix P_m forms a route with skip connections. Eventually, the $n_r = \text{rank}(P_m)$ routes can be generated, and their recurrent lengths and bandwidths are learned automatically. The transition model evolves as

$$\mathbf{s}_t = \mathbf{s}_{t-1} \odot (1 - \mathbf{p}_t) + S(\mathbf{s}_{t-1}, x_t) \odot \mathbf{p}_t \tag{2}$$

and specifically, when used in different recurrent networks, the cell structures can be formulated as follows:

$$\text{RNN}: \mathbf{h}_t = \tanh(U_h \mathbf{h}_{t-1} + W_h \mathbf{x}_t + \mathbf{b}_h)$$

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

$$\text{DRRRNN:}\begin{cases}\tilde{\mathbf{h}}_t = \tanh(U_h\mathbf{h}_{t-1} + W_h\mathbf{x}_t + \mathbf{b}_h)\\ \mathbf{p}_t = G(U_p\mathbf{h}_{t-1} + W_p\mathbf{x}_t + \mathbf{b}_p, \tau)\\ \mathbf{h}_t = \mathbf{h}_{t-1}\odot(1-\mathbf{p}_t) + \tilde{\mathbf{h}}_t \odot \mathbf{p}_t\end{cases} \tag{3}$$

$$\text{LSTM}:\begin{cases}\mathbf{f}_t = \sigma(U_f\mathbf{h}_{t-1} + W_f\mathbf{x}_t + \mathbf{b}_f)\\ \mathbf{i}_t = \sigma(U_i\mathbf{h}_{t-1} + W_i\mathbf{x}_t + \mathbf{b}_i)\\ \mathbf{o}_t = \sigma(U_o\mathbf{h}_{t-1} + W_o\mathbf{x}_t + \mathbf{b}_o)\\ \mathbf{g}_t = \tanh(U_g\mathbf{h}_{t-1} + W_g\mathbf{x}_t + \mathbf{b}_g)\\ \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t\\ \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)\end{cases}$$

$$\text{DRRLSTM}:\begin{cases}\mathbf{f}_t = \sigma(U_f\mathbf{h}_{t-1} + W_f\mathbf{x}_t + \mathbf{b}_f)\\ \mathbf{i}_t = \sigma(U_i\mathbf{h}_{t-1} + W_i\mathbf{x}_t + \mathbf{b}_i)\\ \mathbf{o}_t = \sigma(W_o\mathbf{x}_t + \mathbf{b}_o)\\ \mathbf{g}_t = \tanh(U_g\mathbf{h}_{t-1} + W_g\mathbf{x}_t + \mathbf{b}_g)\\ \mathbf{p}_t = G(U_p\mathbf{h}_{t-1} + W_p\mathbf{x}_t + \mathbf{b}_p, \tau)\\ \mathbf{c}_t = \mathbf{c}_{t-1}\odot(1-\mathbf{p}_t)\\ \quad +[\mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t]\odot \mathbf{p}_t\\ \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)\end{cases} \tag{4}$$

$$\text{GRU}:\begin{cases}\mathbf{r}_t = \sigma(U_r\mathbf{h}_{t-1} + W_r\mathbf{x}_t)\\ \mathbf{z}_t = \sigma(U_z\mathbf{h}_{t-1} + W_z\mathbf{x}_t)\\ \tilde{\mathbf{h}}_t = \tanh(U_h\mathbf{r}_t \odot \mathbf{h}_{t-1} + W_h\mathbf{x}_t)\\ \mathbf{h}_t = \mathbf{h}_{t-1}\odot(1-\mathbf{z}_t) + \tilde{\mathbf{h}}_t \odot \mathbf{z}_t\end{cases}$$

$$\text{DRRGRU}:\begin{cases}\mathbf{r}_t = \sigma(U_r\mathbf{h}_{t-1} + W_r\mathbf{x}_t)\\ \mathbf{p}_t = G(U_p\mathbf{h}_{t-1} + W_p\mathbf{x}_t + \mathbf{b}_p, \tau)\\ \tilde{\mathbf{h}}_t = \tanh(U_h\mathbf{r}_t \odot \mathbf{h}_{t-1} + W_h\mathbf{x}_t)\\ \mathbf{h}_t = \mathbf{h}_{t-1}\odot(1-\mathbf{p}_t) + \tilde{\mathbf{h}}_t \odot \mathbf{p}_t\end{cases} \tag{5}$$

where τ is a temperature parameter. In DRRRNN $\tilde{\mathbf{h}}_t$ is a middle variant and $\mathbf{h}_t$ acts as both the memory state and the hidden output. In DRRLSTM, the four gates $\{\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t, \mathbf{g}_t\} \in \mathbb{R}^{n_2}$ and their inner weights take the same meaning as those in the standard LSTM [7], but $\mathbf{o}_t$ only depends on x_t, which is different from the standard LSTM and aims to guarantee $\mathbf{h}_t$ of multiroute (the reasons are detailed in the first paragraph of Section II-B). $\mathbf{c}_t \in \mathbb{R}^{n_2}$ is the memory state and $\mathbf{h}_t \in \mathbb{R}^{n_2}$ is the output of hidden layer. In DRRGRU, $\tilde{\mathbf{h}}_t \in \mathbb{R}^{n_2}$ and $\mathbf{h}_t \in \mathbb{R}^{n_2}$ take the same meaning as those in DRRRNN, and $\mathbf{r}_t \in \mathbb{R}^{n_2}$ is a reset gate. An example for the cell structure of DRRLSTM is shown in Fig. 1(b) as the structure is more complicated than those of other models. The updating schemes of the state entries in the three models are all based on the pointer $\mathbf{p}_t$, whose elements are binarized by a Gumbel function $G(*)$ or take a soft thresholding, i.e., $\mathbf{p}_t = \sigma(U_p\mathbf{h}_{t-1} + W_p\mathbf{x}_t + \mathbf{b}_p)$, and we will find an appropriate form for each model by ablation analysis in the experiments. In the formulas above, the cell structures of all the models are presented in the form of hard thresholding, which is equivalent to extracting samples according to the probability of $\sigma(U_p\mathbf{h}_{t-1} + W_p\mathbf{x}_t + \mathbf{b}_p)$ ($\sigma(\beta)$ for short, i.e., $\beta = U_p\mathbf{h}_{t-1} + W_p\mathbf{x}_t + \mathbf{b}_p$), and the sample entries follow the Bernoulli distribution B:

$$D_{\beta_i} \sim B(\sigma(\beta_i)) \tag{6}$$

where β_i is the ith entry of β and D is a sample entry. In this article, we adopt a Gumbel function proposed by Maddison *et al.* [21]

$$G(\beta_i, \tau) = \sigma\left(\frac{\beta_i + \log\varepsilon_i - \log(1-\varepsilon_i))}{\tau}\right) \times \varepsilon_i \sim \text{Uniform}(0,1) \tag{7}$$

where ε_i is a noise extracted from an uniform distribution. It has been demonstrated in [21] that

$$P\left(\lim_{\tau\to 0^+} G(\beta_i,\tau) = 1\right) = P(D_{\beta_i} = 1)$$
$$P\left(\lim_{\tau\to 0^+} G(\beta_i,\tau) = 0\right) = P(D_{\beta_i} = 0) \tag{8}$$

when the temperature parameter τ approaches zero and $G(\beta_i, \tau)$ follows the same distribution as that presented in (6). Therefore, the entries of $\mathbf{p}_t$ in (5) can take binary values according to a Binomial distribution. In our experiments, τ is set to be 0.01. The strategy on how to enforce the matrix P_m to be low rank is presented in Section III.

The architectures via dynamic recurrent routing can also contribute to alleviate gradient vanishing (exploding) problem. Suppose that a recurrent network has an objective function $\mathcal{L}$ and a weight parameter W_s, and the gradient can be formulated as

$$\frac{\partial\mathcal{L}}{\partial W_s} = \sum_{j=0}^{T}\frac{\partial\mathcal{L}}{\partial h_T}\frac{\partial h_T}{\partial s_T}\left(\prod_{t=j+1}^{T}\frac{\partial s_t}{\partial s_{t-1}}\right)\frac{\partial s_j}{\partial W_s} \tag{9}$$

where h_t and s_t denote the hidden output and cell state, respectively. The gradient depends on the terms of successive multiplication; note that a derivative of an element-wise multiplication of two vectors is written as a vector,[1] and thus, $(\partial s_t/\partial s_{t-1})$ is a vector. In conventional recurrent networks, the number of multiplications in the longest successive multiplication term equals the sequence length, but in DRRNets, the multiplicative accumulation is mitigated as the cell state s_t may copy its elements [see Fig. 1(a)] and result in that a group of elements in $(\partial s_t/\partial s_{t-1})$ equal to one; in other words, the path length of gradient propagation is shortened actually.

B. Convolution Architecture in Fully Connected Layers

In this section, we propose to use convolution architecture in the fully connected layers of recurrent networks, which are associated with the hidden state $\mathbf{h}_t$. First, structured input data can provide premise for performing convolutional operation on itself. Based on the multiroute memory state, $\mathbf{h}_t$ in DRRRNN and DRRGRU are structured as they act as both memory and hidden states and are propagated forward recurrently; $\mathbf{h}_t$ in DRRLSTM can also be regarded as multiroute vector, the reasons lie in that: 1) the state vector c_t is of multiroute; 2) the operation of tanh activation will not exchange information among different routes; and 3) the pointwise multiplication in $\mathbf{o}_t \odot \tanh(\mathbf{c}_t)$ means that the components of $\tanh(\mathbf{c}_t)$ are modulated by the output gate o_t which is generated from the current input x_t, and this operation will not change the multiroute probability of $\tanh(\mathbf{c}_t)$ as modulation does not affect the source of information in each component.

[1]The demonstration is provided by Westhuizen and Lasenby [22] in page 5.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

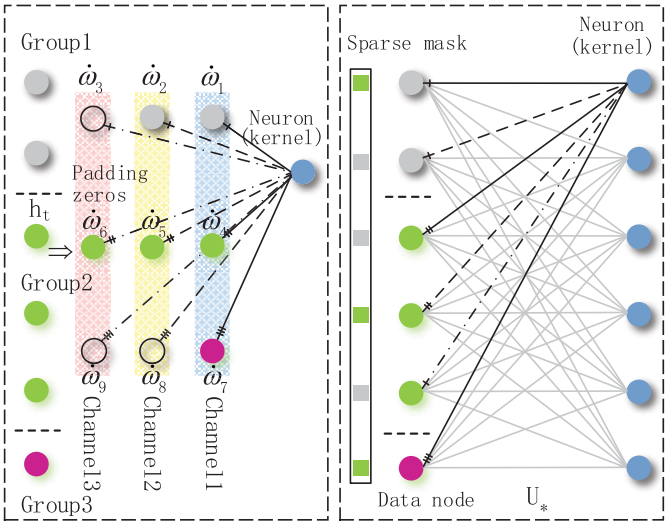


Fig. 2. Illustration of the convolution architecture in fully connected layers. In both subfigures, the nodes on the right side represent convolution kernels, and the black solid lines indicate weights in a perception field, each having different values and marked by different signs. The kernel weights are shared among channels, i.e., the dashed lines share the weights of the solid lines by multiplying a factor $\dot{\omega}_i$. In the right subfigure, a sparse mask acts as a grouping indicator, where each green rectangle indicates a group.

Then, a 1-D convolution network can be used to extract information from $\mathbf{h}_t$ of the RNNs. A custom architecture is presented in the left part of Fig. 2, where the hidden state $\mathbf{h}_t$ is divided into three groups and reshaped into multichannel sequence by zero padding, and a $(1 \times L \times C)$ convolution kernel with $1 \times L$ receptive field and C channels is adopted to perform the convolutional operation. If length L is equal to the number of groups, then the kernel is functionally equivalent to a neuron in fully connected layer. Hence, every neuron in the fully connected layer can be regarded as a specialized 1-D convolutional kernel. However, the problem lies in that this kind of convolution does not make use of parameter sharing mechanism to save parameters. Typically, the convolution network can save parameters by sliding the kernels over input data or further share parameters among different channels within each kernel. Here, we only can take the second method as the analogical receptive fields of the neurons in fully connected layers are fixed to the group number. Specifically, the kernel weights over the first channel are different from each other and then copied to the rest channels (see the left part of Fig. 2).

Next, we can draw low-rank property from the specialized fully connected weights that can be formulated as follows:

$$U_* = \begin{bmatrix} \begin{pmatrix} \omega_{11} & \omega_{11} \\ \omega_{21} & \omega_{21} \\ \vdots & \vdots \\ \omega_{n_2 1} & \omega_{n_2 1} \end{pmatrix} \cdots \begin{pmatrix} \omega_{1n_r} & \omega_{1n_r} \\ \omega_{2n_r} & \omega_{2n_r} \\ \vdots & \vdots \\ \omega_{n_2 n_r} & \omega_{n_2 n_r} \end{pmatrix} \end{bmatrix} \tag{10}$$

where $U_* \in \mathbb{R}^{n_2 \times n_2}$ refers to all of the fully connected weights associated with the grouped $\mathbf{h}_t$ by using a subscript, for instance, the subscript $*$ indicates the four gates ($\{f, i, g, p\}$) in DRRLSTM; the matrix row indicates the weights of one neuron in the fully connected layer (see the right part of Fig. 2), and the matrix column indicates the weights connected with one data node. The former two columns in parenthesis represent the weights connected with the data nodes in the first group. We can see that the columns are repeated within a group; in other words, the weights associated with the first channel are copied to the rest channels. The last two columns in parenthesis represent the weights connected to the data nodes of the last group. n_r is the number of groups and apparently is also the rank of the matrix. Nevertheless, the weight formula is generated by weight copy among the channels within a kernel, which causes two drawbacks: 1) the kernel allocates the same weights on different channels and then decreases its perceptual ability; and 2) the kernel ignores the distinction during weight sharing and a group of importance factors ($[\dot{\omega}_1, \ldots, \dot{\omega}_{n_2}]$) should be multiplied with the corresponding kernel weights in (11), as shown at the bottom of the next page, where n_c indicates the channel number. Thus, a more efficient parameter sharing mechanism is to use a linear combination of the first channel weights to replace direct copy operation, and the coefficients of linear combination are different and trainable for each data node in the rest channels but should be maintained the same among different kernels. It is not hard to deduce that the mechanism is equivalent to perform column elementary transformations on the matrix in (11), which can place different emphasis on the shared weights and will not change the matrix rank. The final formulation of the matrix can be illustrated in (12), as shown at the bottom of the next page, from which we can see that the importance factor $\dot{\omega}_i$ can be replaced with the linear combination coefficients (λ_i and $\tilde{\lambda}_i$).

The following objective is to find a low-rank approximation $\tilde{U}_*$ for the fully connected weight matrix U_*, and $\tilde{U}_*$ has rank equivalent to the number of the groups, i.e., the number of the recurrent routes. Nevertheless, the same coefficients of linear combination among different kernels may limit their perceptual abilities. Hence, we use another matrix V to perceive information more precisely, and V is generated by repeating a vector $\mathbf{v}$, $\mathbf{v} \in \mathbb{R}^{n_2}$, which has different elements and can pay attention to the components in one group at different degrees. Then, we have

$$\begin{aligned} &U_* := \tilde{U}_* + V \\ &\text{s.t. } V = [\mathbf{v}, \ldots, \mathbf{v}], \quad \text{rank}(\tilde{U}_*) = n_r. \end{aligned} \tag{13}$$

An optimization scheme via low-rank regularization is proposed in Section III to achieve the objective by network training.

III. Optimization

Given a sequence X and outputs of the network $Y = \{\mathbf{y}_1, \ldots, \mathbf{y}_T\}$, the objective function can be formulated as

$$\begin{aligned} &\min_{\theta} \mathcal{L}(X, Y; \theta) \\ &\text{s.t. } \text{rank}(P_m) = n_r, \quad \text{rank}(U_c) = n_r \\ &U_c = \begin{cases} \text{DRRRNN}: & [U_h, U_p] \in \mathbb{R}^{n_2 \times 2n_2} \\ \text{DRRLSTM}: & [U_f, U_i, U_g, U_p] \in \mathbb{R}^{n_2 \times 4n_2} \\ \text{DRRGRU}: & [U_r, U_p, U_h] \in \mathbb{R}^{n_2 \times 3n_2} \end{cases} \end{aligned} \tag{14}$$

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

where θ represents a set of the network parameters and U_c is a concatenated matrix and its rank should be forced to be the same as that of the pointer matrix P_m. Inspired by the low-rank constraint strategy presented in [23], which is used to obtain low-rank representation for the network's outputs, we propose an optimization scheme that can impose low-rank constraint on two kinds of matrices and construct approximate matrix decomposition automatically. As presented in [23], a low-rank matrix can be projected onto itself by using a low-rank projection. Then, the objective matrices in our model can be forced to be low rank by introducing auxiliary parameters: $A, B_1, B_2 \in \mathbb{R}^{n_2 \times n_2}$ and a mask vector $\mathbf{m} \in \mathbb{R}^{n_2}$. The augmented objective function can be formulated as

$$\begin{aligned} &\min_{\theta} \mathcal{L}(X, Y; \theta) + \mathcal{L}_1(P_m; A) + \mathcal{L}_2(U_c; B_1, B_2, M) \\ &\text{s.t. } \text{rank}(A) \ll n_2, \quad ||\mathbf{m}||_0 \ll n_2 \end{aligned} \tag{15}$$

where $\mathcal{L}_1$ is a regularizer that can penalize the matrix P_m to be of low rank by using an auxiliary matrix A and $\mathcal{L}_2$ is the other regularizer that forces the matrix U_c to satisfy low-rank property by using B_1, B_2, and M. $||.||_0$ signifies the ℓ_0-norm and can impose sparsity constraint on the mask $\mathbf{m}$. The two regularization terms are illustrated as follows:

$$\begin{aligned} &\mathcal{L}_1(P_m; A) = \frac{1}{b} \sum_{i=1}^{b} \left\| A P_m^{(i)} - P_m^{(i)} \right\|_2^2 \\ &\text{s.t. } \text{rank}(A) \ll n_2 \end{aligned} \tag{16}$$

where $||.||_2$ signifies the ℓ_2-norm, b is the batch size of input sequences, and the superscript i indicates the ith P_M generated on a batch data. There must exist a low-rank identity map A that can project $[P_M^{(1)}, \ldots, P_M^{(b)}] \in \mathbb{R}^{n_2 \times bT}$ onto itself if it is of low rank, and this can be relaxed to a case that $\{P_M^{(1)}, \ldots, P_M^{(b)}\}$ are of low rank. Therefore, this regularizer is to jointly learn a low-rank projection A and penalize the matrices $\{P_M^{(1)}, \ldots, P_M^{(b)}\}$ to be of low-rank. In the training process, A is enforced to be low rank by a hard threshold operator based on singular value decomposition (SVD)

$$\begin{aligned} &\mathcal{L}_2(U_c; B_1, B_2, \mathbf{m}) = ||B_1 \text{diag}(\mathbf{m}) B_2 U_c - U_c||_2^2 \\ &\text{s.t. } ||\mathbf{m}||_0 \ll n_2 \end{aligned} \tag{17}$$

where $\text{diag}(\mathbf{m})$ is a diagonal matrix based on the vector $\mathbf{m}$. If $\mathbf{m}$ is sparse, then $B_1 \text{diag}(M) B_2$ will become a low-rank matrix; B_1 and B_2 are multiplied on both sides of $\mathbf{m}$ in order to eliminate zero rows or columns in the generated matrix. This regularizer is to learn a low-rank projection $B_1 \text{diag}(\mathbf{m}) B_2$ and obtain low rank U_c. In the training process, $\mathbf{m}$ is enforced to be sparse by a step-wise ℓ_0 projection, in which the number of nonzero elements is equivalent roughly to that of groups (see Fig. 2, the mask should strike a balance among sequences in the entire dataset).

Algorithm 1 Optimization Procedure—Stage 1

Input: Input sequences, learning rate η;
Parameter: Network weights θ; mask parameter $\mathbf{s}$;
Output: θ^*;

Initialize $\mathbf{m} = \sigma(\mathbf{s})$, $\mathbf{s} = \mathbf{0}$, $k = r$, $\Delta k = 1$.
while e $\leq$ epoch number **do**
  while i $\leq$ batch number **do**
    // update by gradient descent:
    $\theta = \theta - \eta(\nabla_\theta(\mathcal{L} + \mathcal{L}_1 + \mathcal{L}_2))$
    $A = A - \eta(\nabla_A \mathcal{L}_1)$
    // hard-threshold the rank of A to be k:
    $A = \prod_k^{rank}(A)$
    $[B_1, B_2, \mathbf{s}] = [B_1, B_2, \mathbf{s}] - \eta\ (\nabla_{B_1, B_2, \mathbf{s}} \mathcal{L}_2)$
    // sparsity projection:
    $\mathbf{m} = \sigma(\mathbf{s})$
    Perform ℓ_0 constraint projection on $\mathbf{m}$, generate $\mathbf{m}_{\ell_0}$, s.t. $||\mathbf{m}_{\ell_0}||_0 \leq k$;
  end while
  if θ converges **then**
    // step wise reduction of the sparsity
    $k = k - \Delta k$;
  end if
end while
return θ^* with best accuracy on validation set.

The optimization is split into two stages: training and fine-tuning. The first stage is presented in Algorithm 1, where r is a predefined parameter and k is used in two aspects: 1) hard-thresholding the rank of A and then constrain P_m and 2) constraining sparsity of the mask $\mathbf{m}$ by ℓ_0 projection in which the maximal k elements are preserved while others are reset to zeros. The fact that there exists a low-rank projection via the ℓ_0 norm means that training of DRRNets for each epoch is longer than the vanilla counterparts. The total number of training epochs can be adjusted by setting of r, and as k

$$U_* := \left[\dot{\omega}_1 \begin{pmatrix} \omega_{11} \\ \omega_{21} \\ \vdots \\ \omega_{n_2 1} \end{pmatrix} \cdots \dot{\omega}_{n_c} \begin{pmatrix} \omega_{11} \\ \omega_{21} \\ \vdots \\ \omega_{n_2 1} \end{pmatrix} \cdots \dot{\omega}_i \begin{pmatrix} \omega_{1n_r} \\ \omega_{2n_r} \\ \vdots \\ \omega_{n_2 n_r} \end{pmatrix} \cdots \dot{\omega}_{n_2} \begin{pmatrix} \omega_{1n_r} \\ \omega_{2n_r} \\ \vdots \\ \omega_{n_2 n_r} \end{pmatrix} \right] \tag{11}$$

$$U_* := \left[\begin{pmatrix} \omega_{11} & \lambda_1 \omega_{11} + \cdots + \lambda_{n_r} \omega_{1n_r} \\ \omega_{21} & \lambda_1 \omega_{21} + \cdots + \lambda_{n_r} \omega_{2n_r} \\ \vdots \quad \cdots & \vdots \\ \omega_{n_2 1} & \lambda_1 \omega_{n_2 1} + \cdots + \lambda_{n_r} \omega_{n_2 n_r} \end{pmatrix} \cdots \begin{pmatrix} \omega_{1n_r} & \tilde{\lambda}_1 \omega_{11} + \cdots + \tilde{\lambda}_{n_r} \omega_{1n_r} \\ \omega_{2n_r} & \tilde{\lambda}_1 \omega_{21} + \cdots + \tilde{\lambda}_{n_r} \omega_{2n_r} \\ \vdots \quad \cdots & \vdots \\ \omega_{n_2 n_r} & \tilde{\lambda}_1 \omega_{n_2 1} + \cdots + \tilde{\lambda}_{n_r} \omega_{n_2 n_r} \end{pmatrix} \right] \tag{12}$$

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

TABLE I
RESULTS OF ABLATION ANALYSIS[a]

Model	Layers / hidden units	Parameters (≈)	MNIST (784-long)	Permuted MNIST (784-long)
RNN	1 /128	16.1K	11.9	65.5
RNN+HP_m	1 /128	32.3K	16.1	72.2
RNN+SP_m	1 /128	32.3K	11.3	92.4
RNN+HP_m+$\mathcal{L}_1$	1 /128	32.3K	76.5	38.1
RNN+SP_m+$\mathcal{L}_1$	1 /128	32.3K	**98.6**	**93.3**
LSTM	1 /128	65.0K	98.5	91.0
LSTM+HP_m	1 /128	80.3K	98.8	91.8
LSTM+SP_m	1 /128	80.3K	98.7	91.6
LSTM+HP_m+$\mathcal{L}_1$	1 /128	80.3K	**99.0**	**94.2**
LSTM+SP_m+$\mathcal{L}_1$	1 /128	80.3K	98.9	93.6
GRU	1 /128	48.8K	99.0	91.6
GRU+H	1 /128	48.8K	96.9	86.2
GRU+$\mathcal{L}_1$	1 /128	48.8K	**99.1**	**93.4**
GRU+H+$\mathcal{L}_1$	1 /128	48.8K	98.0	83.6

[a] Bold values indicate the first ranking.

descends from r step by step, an appropriate value of r can reduce the training epochs significantly.

In the second stage, the goal is to reduce memory requirement of the network by using the decoupling strategy. Output of the first stage is a low-rank fully connected parameter matrix U_c^*, which is a subset of θ^*. Then, SVD is performed on U_c^*: $U_c^* = Q\Sigma R^T$. Since U_c^* has rank k, we can obtain an equality as $U_c^* = Q_k \Sigma_k R_k^T$, where Q_k and R_k are the first k columns of Q and R, respectively, and Σ_k is a diagonal matrix and preserves k maximum elements of Σ (i.e., k nonzero elements in this low-rank case). Subsequently, U_c^* is decoupled as

$$U_c^* = \left(Q_k\sqrt{\Sigma_k}\right)\left(\sqrt{\Sigma_k}R_k^T\right) \tag{18}$$

and we use the multiplication of two matrices $\Phi \in \mathbb{R}^{n_2\times k}$ and $\Psi \in \mathbb{R}^{k\times 4n_2}$ to replace the matrix U_c^* and initialize them as

$$\begin{aligned}\Phi &= Q_k\sqrt{\Sigma_k}\\ \Psi &= \sqrt{\Sigma_k}R_k^T\end{aligned} \tag{19}$$

after several fine-turning epochs, we can get a network that has used less parameters while maintaining the performance.

IV. EXPERIMENTS

We conduct an ablation analysis on the MNISTs classification task and then evaluate the proposed model on three diverse applications: sequential image classification, language modeling, and speaker recognition.

A. Ablation Analysis

The MNIST digits [24] are widely used to test recurrent neural networks (RNNs), and thus, we adopt it for ablation analysis on our proposed models. The MNIST dataset is split into training, validation, and testing sets according to the default fraction in TensorFlow (55 000, 5000, and 10 000 images, respectively), and its images are flattened into 784-long sequences. Also, a permuted MNIST [25] is utilized, which is generated by permuting the pixels of the MNIST digits in the same random order and then creates longer term dependencies. The testing results of our models with different components are presented in Table I and the accuracy curves over training epochs are shown in Fig. 3, and all of them are averaged over four independent runs. As for DRRRNN, we compare its four kinds of variations with the vanilla RNN, in which RNN + HP_m indicates the RNN network with hard thresholding pointer matrix formulating different forward pass routes, RNN + SP_m indicates that with soft thresholding, RNN + HP_m + $\mathcal{L}_1$ means imposing a low-rank constraint on the hard thresholding P_m, and the last variation RNN + SP_m + $\mathcal{L}_1$ means that the soft thresholding P_m is penalized to be of low rank. All the models fail on MNIST expect RNN + SP_m + $\mathcal{L}_1$ that improves substantially with the help of both soft thresholding P_m and low-rank constraint, while on Permuted MNIST, RNNs with soft thresholding perform much better than that with hard thresholding. Thus, we choose to use soft thresholding DRRRNN in Sections IV-A and IV-C. As for DRRLSTM, LSTM + HP_m has the most rough ascent on MNIST, but it can be smoothed by low-rank constraint (LSTM + HP_m + $\mathcal{L}_1$), on the more difficult dataset Permuted MNIST, LSTM + HP_m + $\mathcal{L}_1$ achieves the highest accuracy eventually, and thus, we choose to use hard thresholding in the following test of DRRLSTM. As for DRRGRU, the variation GRU + H represents a conventional GRU with hard thresholding on the pointer matrix, and GRU + H + $\mathcal{L}_1$ indicates a further low-rank constraint on P_m. Actually, GRU can be regarded as a model with soft thresholding, which behaves worse on both MNIST and Permuted MNIST when taking hard thresholding (GRU + H) instead. GRU + $\mathcal{L}_1$ improves substantially over GRU when low rank is imposed on the pointer matrix. Consequently, we choose soft thresholding in the DRRGRU model for the following testing tasks. In addition, in all of the variations with low-rank constraint (+$\mathcal{L}_1$), the hyperparameter r used in the optimization procedure (Algorithm 1) is set to 32.

1) Sequential Image Classification: The MNIST digits and CIFAR10 are adopted here. In experimental settings, the first dataset is split in the same proportion as that used in ablation analysis; the second dataset consists of 60 000 32 × 32 three-channel images in ten classes, with 6000 images per class, and is divided into five training batches and one testing batch, each with 10 000 images. The images are reshaped into 1024-long sequences with three channels. The comparative results are presented in Table II. For the models of Standard LSTM, GRU, JANET, Skip LSTM, Dilated LSTM, FastGRNN, and DRRNets, training is performed for 100 epochs using the Adam [26] optimizer, with a learning rate of 0.001 and a batch size of 200. Especially, in DRRNets, 70 epochs are allotted for training and 30 for fine-tuning. All of the results are obtained over four independent runs. Then, for the rest models, we cited the results directly from their articles. Models are compared by using roughly the same parameter numbers that are listed in the table. In the standard LSTM comprising four gates, the total number of parameters is $4(n_1n_2 + n_2^2 + n_2)$. As for DRRLSTM in DRRNets, parameters used in testing stage amount to less than $5(n_1n_2+2rn_2+n_2)$, where r is a predefined rank used in Algorithm 1 and $2rn_2$ provides an upper bound for the parameters in Ψ and Φ as the rank of $\Psi * \Phi$ (U_c^*) is declined from and will be less than r. In two settings of DRRNets (1/128;1/512), r takes value of 32, and during

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

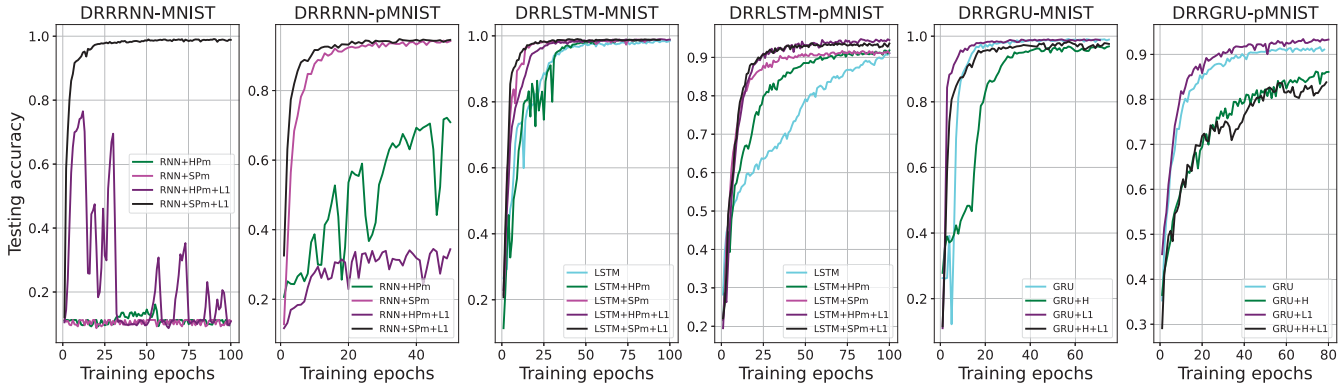


Fig. 3. Testing curves of ablation analysis.

TABLE II
RESULTS OF TESTING ACCURACIES

	Model	Layers / hidden units	Parameters[a] (≈)	MNIST (784-long)	Train time(hr)	Permuted MNIST (784-long)	CIFAR10 (1024×3-long)
Run by us	Standard LSTM [7]	1 /128	65.0K	98.5	6.25	91.0	55.1
	GRU	1 /128	48.8K	**99.0**	5.84	91.6	55.3
	JANET [22]	1 /128	32.5K	**99.0**	5.78	92.5	55.7
	Skip LSTM [15]	1 /100	41.1K	98.2	8.63	90.5	60.7
	Dilated LSTM [13]	9 / 24	39.2K	98.9	3.94	90.7	59.4
	FastGRNN [27]	1 /128	17.6K	98.5	5.61	90.3	58.0
	DRRRNN (r=32)	1 /128	7.5|7.5|8.0K	98.4	6.19	91.9	60.5
	DRRLSTM (r=32)	1 /128	18.8|20.0|21.3K	98.9	7.43	93.9	64.5
	DRRGRU (r=32)	1 /128	11.3|12.0|12.8K	**99.0**	7.24	94.8	**65.2**
Cited results	FastGRNN [27]	−−	6K	98.2	15.10†	−−	−−
	FastGRNN-LSQ [27]	−−	14K	98.7	12.57†	−−	−−
	oRNN [30]	−−	11K	97.2	−−	−−	−−
	AntisymmetricRNN [28]	1/128(256)[b]	10K(36K)[b]	98.0	8.61‡	**95.8**	58.7
	AntisymmetricRNNw/gating[28]	1/128(256)[b]	10K(37K)[b]	98.8	−−	93.1	62.2
	ERNN(K=1) [29]	−−	4K (8K)[c]	97.7	2.83‡	95.6	−−
	Zoneout [31]	1 /100	42K	−−	−−	93.1	−−
	Vanilla RNN [13]	1 /256	68K	−−	−−	71.6	−−
Run by us	DRRRNN (r=100)	1 /128	7.5|8.0K	98.7	18.52	92.5	−−
	DRRLSTM (r=100)	1 /128	18.8|20.0K	98.9	22.14	94.2	−−
	DRRGRU (r=100)	1 /128	11.3|12.0K	**99.0**	21.42	95.6	−−
Run by us	Standard LSTM [7]	1 /256	270.0K	98.2	6.30	91.7	56.1
	JANET [22]	1 /512	514.0K	99.1	5.92	93.3	57.9
	Skip LSTM [15]	1 /200	160.4K	98.6	9.17	92.9	62.4
	Dilated LSTM [13]	9 / 50	173.0K	98.9	4.05	95.4	60.2
	DRRRNN (r=32)	1 /512	47.0|47.5|49.7K	99.0	6.27	93.7	62.7
	DRRLSTM (r=32)	1 /512	122.5|127.5|128.8K	99.0	7.56	95.2	68.2
	DRRGRU (r=32)	1 /512	73.5|74.3|76.1K	99.1	7.34	**95.8**	**68.4**
Cited results	Full uRNN [32]	1 /512	270.0K	97.5	−−	94.1	−−
	tLSTM [33]	3 /100	195.7K	**99.2**	−−	94.9	−−

[a] For DRRNets, as the ranks of full connected layers are different on the three datasets, the parameter numbers on MNIST, Permuted MNIST and CIFAR10 in testing stage are listed separately.
[b] 128 and 256 hidden units are adopted on MNISTs and CIFAR10 respectively, and result in 10K and 36K total parameters; [c] 4K and 8K parameters are used for MNIST and Permuted MNIST respectively.
† The train time are cited results and the models are trained on an Nvidia Tesla P40 GPU [27]; ‡the models are trained on Nvidia GTX 1080 GPU [29].

training, the rank decreases to a region of $14 \sim 16$, which can further reduce the total parameter numbers. For instance, DRRRNN ($r = 32$) actually has roughly 8k parameters after training when its rank declines from $r = 32$ to 16. On the whole, DRRNets behave at roughly the same level as the FastGRNN [27], AntisymmetricRNN [28], and ERNN [29] on MNISTs and outperform other comparative models by using the minimum parameters. On CIFAR10, DRRNets achieve the best performance among all the models apparently.

We also conduct experiments on DRRNets with larger hyperparameter r, and when r is set as 100, the weight rank of the pointer matrix and fully connected layers is reduced from a larger initial value, which takes more training epochs and consequently needs more training time. Thus, 300 epochs for training are used in this setting, and the results on MNISTs are presented in Table II, from which we can see that DRRNets behave slightly better than those using $r = 32$. The models run by us are trained on an Nvidia GTX 1080Ti GPU with CUDA 10 and cuDNN 7.4 on a machine with Intel i7-6800K 3.40-GHz CPU with 12 cores, and the train time of the models on MNIST is listed in a column of Table II.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CHARACTER-LEVEL LANGUAGE MODELING ON THE PTB

Model	Layers / hidden units	Parameters[a] (≈)	Evaluation BPC
Standard LSTM [7]	1 / 1k	4.25M	1.31
GRU [13]	1 / 1k	3.19M	1.32
Recurrent BN-LSTM [35]	1 / 1k	4.25M†	1.32
Recurrent dropout LSTM [36]	1 / 1k	4.25M	1.30
Zoneout [31]	1 / 1k	4.25M	1.27
HyperNetworks [37]	1 / 1k	4.91M	1.26
LayerNorm HM-LSTM [12]	3 /512	5.01M†	**1.24**
Dilated LSTM [13]	5 /256	1.90M	1.31
DRRRNN	1 / 1k	0.4M	1.37
DRRLSTM	1 / 1k	1.96M	**1.24**
DRRGRU	1 / 1k	1.18M	1.26

[a] The parameter number with † is estimated based on our understanding of the model.

2) Language Modeling: We further investigate language modeling on the Penn tree bank (PTB) and WikiText-2 [34] datasets. PTB contains 5059k characters for training, 396k for validation, and 446k for testing, and when used in a word-level language modeling, it contains 888k words for training, 70k for validation, and 79k for testing, with a vocabulary size of 10k; WikiText-2 is sourced from Wikipedia articles and features a vocabulary of over 30k words. The language modeling task is to predict the next character or word based on the previous information by training a network, and the performances are evaluated by the criteria of bits-per-character (BPC) and perplexity (PPL).

In character-level language modeling, our models utilize the Adam optimizer with the learning rate 0.001 and the batch size is set to 128. The data are split into sequences with length of 100 by following the common settings in previous works [12], [35], [37], and the comparisons with existing results are presented in Table III. The total number of parameters in DRRLSTM (1/1 k) is roughly 1.96 M when the hyperparameter r is set to 256 and the rank of U_c^* is about 200, which may differ slightly in different runs. DRRLSTM ranks first in BPC scores by using the minimum parameters in the testing stage.

Then, in the word-level language modeling, we follow the experimental settings used in AWD-LSTM model [34], i.e., a weight-dropped LSTM using averaged stochastic gradient method (AvSGD). AWD-RNN comprises three RNN layers with hidden size of 2k, and AWD-GRU has three GRU layers with hidden size of 1150, which is the same as that used in AWD-LSTM. Then, we replace the top layers with DRRNets that are aimed to collect information from the lower layers and form dynamic routes. In all the models, the embedding size is set to 400, and dropout rates (0.4, 0.3, 0.4, and 0.1) are applied for dropout on the word vectors, the output between LSTM layers, the output of the final layer, and embedding dropout, respectively. The comparisons with other cited results are listed in Tables IV and V, from which we can see that DRRNets have made improvement over AWD-networks by introducing low-rank regularization. In addition, for comparison with the regularized LSTM [46] which uses two layers, we conduct experiments with DRRNets by using one LSTM layer and one DRRLSTM layer, each having 1560 units. The result demonstrates that our model outperforms LSTM [46] on Wiki-text2 by using less parameters. When expanding the hidden units, the model (35 M parameters with 1800 units) behaves much better and gets close to the result of Mogrifier [47], which has applied Mixture of Softmaxes [48] (denoted with MoS) and state dropout in the recurrent cell.

TABLE IV
WORD-LEVEL LANGUAGE MODELING ON THE PTB

Model	Parameters (≈)	PPL
LSTM (medium) [38]	20M	82.7
LSTM (large) [38]	66M	78.4
GRU [39]	13M	92.5
Variational LSTM [40]	20M	78.6
CharCNN [41]	19M	78.9
Pointer Sentinel-LSTM [42]	21M	70.9
Variational LSTM + augmented loss [43]	24M	73.2
Variational LSTM + augmented loss [43]	51M	68.5
Variational RHN [44]	23M	65.4
NAS Cell [45]	25M	64.0
NAS Cell [45]	54M	62.4
4-layer skip connection LSTM [46]	24M	58.3
AWD-RNN: 3-layer vanilla RNN	18M	103.6
AWD-LSTM: 3-layer LSTM [34]	24M	57.3
AWD-GRU: 3-layer GRU	18M	82.8
DRRNets: 2-layer RNN + 1-layer DRRRNN	15M[a]	91.2
DRRNets: 2-layer LSTM + 1-layer DRRLSTM	21M	**56.1**
DRRNets: 2-layer GRU + 1-layer DRRGRU	16M	78.4

[a] Parameter saving only exists in the top layer.

TABLE V
WORD-LEVEL LANGUAGE MODELING ON THE WIKI-TEXT2

Model	Parameters (≈)	PPL
LSTM [28]	24M	65.9
2-layer skip connection LSTM [28]	24M	65.9
Temporal Convolution Network (TCN) [2]	17M	110.3
Variational LSTM [12]	28M	87.7
Variational LSTM (tied) + augmented loss [12]	28M	87.0
AWD-RNN: 3-layer vanilla RNN	26M	108.7
AWD-LSTM: 3-layer LSTM [30]	33M	65.8
AWD-GRU: 3-layer GRU	27M	92.4
DRRNets: 2-layer RNN+1-layer DRRRNN	23M	99.4
DRRNets: 2-layer LSTM+1-layer DRRLSTM	30M	64.6
DRRNets: 2-layer GRU+1-layer DRRGRU	25M	88.5
DRRNets[a]: 1-layer LSTM+1-layer DRRLSTM	24M	64.1
DRRNets[b]: 1-layer LSTM+1-layer DRRLSTM	35M	<u>57.2</u>
Mogrifier (d2, MoS2) [29]	35M	**56.6**

[a] 1560 hidden units are used in both layers.
[b] 1800 hidden units are used in both layers.

3) Speaker Recognition:

a) TM-speech: As throat microphone is a nonacoustic sensor that collects vibration signals directly from the human throat skin, the throat microphone speech (TM-Speech) [49] is feasible about used in secure communication or for conversation in intensive noise environments such as extreme sports. Signal processing, such as speaker recognition or voice conversion on the speech data, should not only consider the frame-wised information but also the long-term dependencies in the context of acoustic trajectory; moreover, learning directly from the features of spectral magnitudes [50] poses a

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
TESTING ACCURACY ON TM-SPEECH

Model	Parameters[a] (≈)	TM-Speech[b] (1280-long)
Standard LSTM [7] (1/128)	128.5K	79.2
JANET [22] (1/256)	193.0K	80.6
Skip LSTM [15] (1/200)	194.7K	20.7
Dilated LSTM [13] (9/50)	193.2K	74.7
DRRRNN (1/128)	47.3K	78.7
DRRLSTM (1/128)	96.5K	**82.1**
DRRGRU (1/128)	71.3K	80.9

[a] Parameter numbers differ from those in Table II due to different input sizes;
[b] the features are stored into 1280-long sequences with initial zero vectors.

TABLE VII
TESTING ACCURACY ON AISHELL-V1-SPEECH

Model	Parameters (≈)	Aishell-v1
4DNN [52]	10.04M	93.1
1CNN + 3RNN	2.70M	86.9
1CNN + 3DRRRNN	3.64M	91.5
1CNN + 3LSTM	9.47M	96.8
1CNN + 3DRRLSTM	8.28M	97.2
Deep Speaker[a] [51]	7.21M	95.6
1CNN + 3DRRGRU	5.24M	96.4

[a] 1CNN + 3GRU.

difficult challenge for models, as the features are low discriminating due to severe loss of high-frequency components. In this section, we perform an evaluation by recognition accuracy on these features. The speech corpus is established as follows: five female speakers are required to read 200 sentences, which lasts for about 5 s and may be different for each speaker, the speeches are recorded by a throat microphone simultaneously; and then, the collected utterances are split into training, validation, and testing sets, which contain 160, 20, and 20 utterances, respectively, per speaker. Next, the utterances are segmented into frames, with length of 32 ms and frameshift of 10 ms, and 129-D spectral magnitudes are extracted on the frames. Finally, the sequential data of features are stored into 1280-long sequences by filling zeros and fed into the models as inputs. The comparative results are presented in Table VI, we can see that Custom RNN and Skip LSTM fail in this task, and our models can achieve high accuracies while using less parameters than others.

b) Aishell-v1-speech: Aishell-v1[2] is an open-source Chinese Mandarin speech corpus and 178 h long, of which utterances contain 11 domains, including smart home, autonomous driving, and industrial production; 400 speakers from different accent areas in China were invited to participate in the recording. The corpus is divided into training, validation, and testing sets, which comprise 85%, 5%, and 10% utterances, respectively, per speaker. In order to achieve good performance in this task, we construct neural architecture following the settings proposed in Deep Speaker [51], i.e., one convolution layer, three GRU layers, and one dense layer are used for recognition on the FBank features extracted from the utterances, where 64 convolution kernels with $[5, 5]$ kernel size and $[2, 2]$ stride size are used, the hidden sizes of GRU layer are set to 512, and the dense layer also has 512 units. We replace the GRU layers with our DRRNets and the standard LSTM for comparison while maintaining the same sizes of layers, and we also implement an algorithm proposed by Variani *et al.* [52], where four DNN layers with units of $[512, 512, 512, 400]$ are adopted in the network.

The comparative results are presented in Table VII. The parameter numbers are listed in the middle column, and for the model of 4DNN, it receives FBank features as inputs that are reshaped from [299, 64] to [19 136, 1], where 299 is the frame number and 64 is the adopted number of frequency points; thus, we can get a total parameter number of 10.04 M. Others perceive FBank features by a convolution layer that outputs 2048-long intermediate vectors that are reshaped from [32, 64], where 32 is the feature dimension halved by strided convolution and 64 is the number of convolution kernels. For the model of 1CNN + 3DRRRNN, as the input size ($n_1 = 2048$ in the bottom layer) is much bigger than the hidden size ($n_2 = 512$), the weights related to inputs [U_* in (3)] hold a majority of parameters and the parameter saving on the weights related to hidden state [W_* in (3)] is not very significant. When a gate (p_t) is added in DRRRNN, the model has more parameters than 1CNN + 3RNN, but compared with 1CNN + 3DRRRNN without low-rank decoupling mechanism (4.45 M), it has saved 0.81 M parameters. In the table, the model of 1CNN + 3RNN behaves weakly than the Deep Speaker and 1CNN + 3 LSTM outperforms it. All the models improve in accuracy rate by using DRRNets and parameters are reduced in those using DRRLSTM and DRRGRU.

[2]http://www.aishelltech.com/kysjcp

V. CONCLUSION

In this article, we propose dynamic recurrent routing neural networks (DRRNets) in order to tackle the issues, such as complex dependencies and gradient vanishing in RNNs. The proposed DRRNets can: 1) exploit the low-rank property of the routing pointer matrix and then generate routes adaptively for different dependencies and 2) reduce redundant parameters significantly and improve the performance by finding low-rank approximation for the fully connected layers based on the inner structure of the cell state. We propose an optimization algorithm for training the network and evaluate the effectiveness of the model in diverse application tasks, including image classification, language modeling, and speaker recognition. The experimental results demonstrate that our model can achieve the state-of-the-art performance while using less parameters than its counterparts.

REFERENCES

[1] K. Shuang, R. Li, M. Gu, J. Loo, and S. Su, "Major-minor long short-term memory for word-level language model," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 10, pp. 3932–3946, Oct. 2020.
[2] M. Dong and C. Xu, "On retrospecting human dynamics with attention," in *Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI)*, Macao, China, S. Kraus, Ed., Aug. 2019, pp. 708–714, doi: 10.24963/ijcai.2019/100.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

[3] B. Zhang, D. Xiong, J. Xie, and J. Su, "Neural machine translation with GRU-gated attention model," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 11, pp. 4688–4698, Nov. 2020.

[4] K. Xu *et al.*, "Show, attend and tell: Neural image caption generation with visual attention," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 2048–2057.

[5] O. Vinyals, A. Toshev, S. Bengio, and D. Erhan, "Show and tell: A neural image caption generator," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 3156–3164.

[6] D. W. Otter, J. R. Medina, and J. K. Kalita, "A survey of the usages of deep learning for natural language processing," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 2, pp. 604–624, Feb. 2021.

[7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] K. Greff, R. K. Srivastava, J. Koutnìk, B. R. Steunebrink, and J. Schmidhuber, "LSTM: A search space odyssey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2222–2232, Oct. 2017.

[9] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, A. Moschitti, B. Pang, and W. Daelemans, Eds. Doha, Qatar: ACL, Oct. 2014, pp. 1724–1734. [Online]. Available: https://dblp.org/rec/conf/emnlp/ChoMGBBSB14.bib, doi: 10.3115/v1/d14-1179.

[10] A. Graves *et al.*, "Hybrid computing using a neural network with dynamic external memory," *Nature*, vol. 538, no. 7626, pp. 471–476, 2016.

[11] D. Neil, M. Pfeiffer, and S.-C. Liu, "Phased LSTM: Accelerating recurrent network training for long or event-based sequences," in *Proc. Adv. Neural Inf. Process. Syst. 29, Annu. Conf. Neural Inf. Process. Syst.*, D. D. Lee, M. Sugiyama, U. von Luxburg, I. Guyon, and R. Garnett, Eds. Barcelona, Spain, Dec. 2016, pp. 3882–3890. [Online]. Available: https://proceedings.neurips.cc/paper/2016/hash/5bce843dd76db8c939d5323dd3e54ec9-Abstract.html and https://dblp.org/rec/conf/nips/NeilPL16.bib

[12] J. Chung, S. Ahn, and Y. Bengio, "Hierarchical multiscale recurrent neural networks," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*. Toulon, France: OpenReview.net, Apr. 2017. [Online]. Available: https://openreview.net/forum?id=S1di0sfgl and https://dblp.org/rec/conf/iclr/ChungAB17.bib

[13] S. Chang *et al.*, "Dilated recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 77–87.

[14] A. W. Yu, H. Lee, and Q. V. Le, "Learning to skim text," in *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, vol. 1, R. Barzilay and M. Kan, Eds. Vancouver, BC, Canada: Association for Computational Linguistics, Jul./Aug. 2017, pp. 1880–1890, doi: 10.18653/v1/P17-1172.

[15] V. Campos, B. Jou, X. Giró-i-Nieto, J. Torres, and S.-F. Chang, "Skip RNN: Learning to skip state updates in recurrent neural networks," 2017, *arXiv:1708.06834*. [Online]. Available: http://arxiv.org/abs/1708.06834

[16] J. Chen and N. S. Chaudhari, "Segmented-memory recurrent neural networks," *IEEE Trans. Neural Netw.*, vol. 20, no. 8, pp. 1267–1280, Aug. 2009.

[17] Y. LeCun, F. J. Huang, and L. Bottou, "Learning methods for generic object recognition with invariance to pose and lighting," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, vol. 2, Jun. 2004, pp. 97–104.

[18] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[19] H. Sak, A. Senior, and F. Beaufays, "Long short-term memory recurrent neural network architectures for large scale acoustic modeling," in *Proc. INTERSPEECH*, 2014, pp. 338–342.

[20] A. V. M. Barone, "Low-rank passthrough neural networks," in *Proc. Workshop Deep Learn. Approaches Low-Resource NLP*. Melbourne, VIC, Australia: Association for Computational Linguistics, Jul. 2018, pp. 77–86. [Online]. Available: https://www.aclweb.org/anthology/W18-3410

[21] C. J. Maddison, A. Mnih, and Y. W. Teh, "The concrete distribution: A continuous relaxation of discrete random variables," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*. Toulon, France: OpenReview.net, Apr. 2017. [Online]. Available: https://openreview.net/forum?id=S1jE5L5gl and https://dblp.org/rec/conf/iclr/MaddisonMT17.bib

[22] J. van der Westhuizen and J. Lasenby, *The Unreasonable Effectiveness of the Forget Gate*, document arXiv:1804.04849v3, Sep. 2018.

[23] A. Sanyal, V. Kanade, and P. H. S. Torr, *Low Rank Structure of Learned Representations*, document arXiv:1804.07090v1, Apr. 2018. [Online]. Available: http://arxiv.org/abs/1804.07090

[24] Y. LeCun, L. Bottou, L. Bottou, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[25] M. Arjovsky, A. Shah, and Y. Bengio, "Unitary evolution recurrent neural networks," in *Proc. 33rd Int. Conf. Mach. Learn. (ICML)*, New York, NY, USA, Jun. 2016, pp. 1120–1128. [Online]. Available: http://jmlr.org/proceedings/papers/v48/arjovsky16.html

[26] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, Y. Bengio and Y. LeCun, Eds. San Diego, CA, USA, May 2015. [Online]. Available: http://arxiv.org/abs/1412.6980 and https://dblp.org/rec/journals/corr/KingmaB14.bib

[27] A. Kusupati, M. Singh, K. Bhatia, A. Kumar, P. Jain, and M. Varma, "FastGRNN: A fast, accurate, stable and tiny kilobyte sized gated recurrent neural network," in *Advances in Neural Information Processing Systems*, vol. 31, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds. Red Hook, NY, USA: Curran Associates, 2018, pp. 9017–9028. [Online]. Available: https://proceedings.neurips.cc/paper/2018/file/ab013ca67cf2d50796b0c11d1b8bc95d-Paper.pdf

[28] B. Chang, M. Chen, E. Haber, and E. H. Chi, "AntisymmetricRNN: A dynamical system view on recurrent neural networks," in *Proc. 7th Int. Conf. Learn. Represent. (ICLR)*. New Orleans, LA, USA: OpenReview.net, May 2019. [Online]. Available: https://openreview.net/forum?id=ryxepo0cFX and https://dblp.org/rec/conf/iclr/ChangCHC19.bib

[29] A. Kag, Z. Zhang, and V. Saligrama, "RNNs incrementally evolving on an equilibrium manifold: A panacea for vanishing and exploding gradients?" in *Proc. 8th Int. Conf. Learn. Represent. (ICLR)*. Addis Ababa, Ethiopia: OpenReview.net, Apr. 2020. [Online]. Available: https://openreview.net/forum?id=HylpqA4FwS and https://dblp.org/rec/conf/iclr/KagZS20.bib

[30] Z. Mhammedi, A. D. Hellicar, A. Rahman, and J. Bailey, "Efficient orthogonal parametrisation of recurrent neural networks using householder reflections," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, in Proceedings of Machine Learning Research, vol. 70, D. Precup and Y. W. Teh, Eds. Sydney, NSW, Australia: PMLR, Aug. 2017, pp. 2401–2409. [Online]. Available: http://proceedings.mlr.press/v70/mhammedi17a.html and https://dblp.org/rec/conf/icml/MhammediHRB17.bib

[31] D. Krueger *et al.*, "Zoneout: Regularizing RNNs by randomly preserving hidden activations," 2016, *arXiv:1606.01305*. [Online]. Available: http://arxiv.org/abs/1606.01305

[32] S. Wisdom, T. Powers, J. Hershey, J. Le Roux, and L. Atlas, "Full-capacity unitary recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 4880–4888.

[33] Z. He, S. Gao, L. Xiao, D. Liu, H. He, and D. Barber, "Wider and deeper, cheaper and faster: Tensorized LSTMs for sequence learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.

[34] S. Merity, N. S. Keskar, and R. Socher, "Regularizing and optimizing LSTM language models," 2017, *arXiv:1708.02182*. [Online]. Available: http://arxiv.org/abs/1708.02182

[35] T. Cooijmans, N. Ballas, C. Laurent, C. C. Gülçehre, and A. C. Courville, "Recurrent batch normalization," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, Apr. 2017, pp. 1–13. [Online]. Available: https://openreview.net/forum?id=r1VdcHcxx

[36] S. Semeniuta, A. Severyn, and E. Barth, "Recurrent dropout without memory loss," 2016, *arXiv:1603.05118*. [Online]. Available: http://arxiv.org/abs/1603.05118

[37] D. Ha, A. Dai, and Q. V. Le, "HyperNetworks," 2016, *arXiv:1609.09106*. [Online]. Available: http://arxiv.org/abs/1609.09106

[38] W. Zaremba, I. Sutskever, and O. Vinyals, *Recurrent Neural Network Regularization*, doucment arXiv:1409.2329, Sep. 2014. [Online]. Available: http://arxiv.org/abs/1409.2329

[39] S. Bai, J. Z. Kolter, and V. Koltun, *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*, document arXiv:1803.01271, Mar. 2018. [Online]. Available: http://arxiv.org/abs/1803.01271

[40] Y. Gal and Z. Ghahramani, "A theoretically grounded application of dropout in recurrent neural networks," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, D. D. Lee, M. Sugiyama, U. von Luxburg, I. Guyon, and R. Garnett, Eds., Barcelona, Spain, Dec. 2016, pp. 1019–1027.

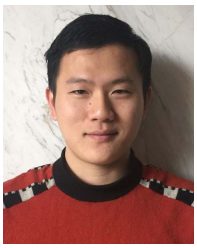
[41] Y. Kim, Y. Jernite, D. Sontag, and A. M. Rush, "Character-aware neural language models," 2015, *arXiv:1508.06615*. [Online]. Available: http://arxiv.org/abs/1508.06615

[42] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer Sentinel mixture models," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, Apr. 2017, pp. 1–13. [Online]. Available: https://openreview.net/forum?id=Byj72udxe

[43] H. Inan, K. Khosravi, and R. Socher, "Tying word vectors and word classifiers: A loss framework for language modeling," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, Apr. 2017, pp. 1–13. [Online]. Available: https://openreview.net/forum?id=r1aPbsFle

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.

[44] J. G. Zilly, R. K. Srivastava, J. Koutník, and J. Schmidhuber, "Recurrent highway networks," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, in Proceedings of Machine Learning Research, vol. 70, D. Precup and Y. W. Teh, Eds., Sydney, NSW, Australia, Aug. 2017, pp. 4189–4198. [Online]. Available: http://proceedings.mlr.press/v70/zilly17a.html
[45] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, Apr. 2017, pp. 1–16. [Online]. Available: https://openreview.net/forum?id=r1Ue8Hcxg
[46] G. Melis, C. Dyer, and P. Blunsom, "On the state of the art of evaluation in neural language models," in *Proc. 6th Int. Conf. Learn. Represent. (ICLR)*, Vancouver, BC, Canada, Apr./May 2018, pp. 1–10. [Online]. Available: https://openreview.net/forum?id=ByJHuTgA-
[47] G. Melis, T. Kočiský, and P. Blunsom, "Mogrifier LSTM," 2019, *arXiv:1909.01792*. [Online]. Available: http://arxiv.org/abs/1909.01792
[48] Z. Yang, Z. Dai, R. Salakhutdinov, and W. W. Cohen, "Breaking the softmax bottleneck: A high-rank RNN language model," in *Proc. 6th Int. Conf. Learn. Represent. (ICLR)*. Vancouver, BC, Canada: OpenReview.net, Apr./May 2018. [Online]. Available: https://openreview.net/forum?id=HkwZSG-CZ and https://dblp.org/rec/conf/iclr/YangDSC18.bib
[49] T. Toda, M. Nakagiri, and K. Shikano, "Statistical voice conversion techniques for body-conducted unvoiced speech enhancement," *IEEE Trans. Audio, Speech, Language Process.*, vol. 20, no. 9, pp. 2505–2517, Nov. 2012.
[50] K. Paliwal, B. Schwerin, and K. Wójcicki, "Speech enhancement using a minimum mean-square error short-time spectral modulation magnitude estimator," *Speech Commun.*, vol. 54, no. 2, pp. 282–305, Feb. 2012.
[51] C. Li *et al.*, "Deep speaker: An end-to-end neural speaker embedding system," 2017, *arXiv:1705.02304*. [Online]. Available: http://arxiv.org/abs/1705.02304
[52] E. Variani, X. Lei, E. McDermott, I. L. Moreno, and J. Gonzalez-Dominguez, "Deep neural networks for small footprint text-dependent speaker verification," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2014, pp. 4052–4056.



Dongjing Shan received the B.Eng. degree in computer science from Beijing Institute of Technology, Beijing, China, in 2008, the M.S. degree in information science and technology from Peking University, Beijing, in 2013, and the Ph.D. degree in signal processing from the Speech Processing Laboratory, Army Engineering University, Nanjing, China, in 2020.

He is currently a Researcher with the Laboratory of Intelligent Information Processing, Army Engineering University, Chongqing, China. His research interests include machine learning and deep learning with applications to image processing and speech recognition.



Yong Luo (Member, IEEE) received the B.E. degree in computer science from the Northwestern Polytechnical University, Xi'an, China, in 2009, and the D.Sc. degree from the School of Electronics Engineering and Computer Science, Peking University, Beijing, China, in 2014.

He is currently a Professor with the School of Computer Science, Wuhan University, Wuhan, China. His research interests are primarily on machine learning and data mining with applications to visual information understanding and analysis. He has authored or coauthored over 40 papers in top journals and prestigious conferences, including IEEE T-PAMI, IEEE T-NNLS, IEEE T-IP, IEEE T-KDE, IEEE T-MM, WWW, IJCAI, AAAI, CIKM, ICDM, and ICME.

Dr. Luo received the IEEE GLOBECOM 2016, the IEEE ICME 2019, and the IEEE VCIP 2019 Best Paper Awards, and was nominated as the IJCAI 2017 Distinguished Best Paper Award.



Xiongwei Zhang received the Ph.D. degree in signal and information processing from Nanjing Institute of Communications Engineering, Nanjing, China, in 1992.

He is currently a Professor with the Speech Processing Laboratory, Army Engineering University, Nanjing. His research interests include speech signal processing, machine learning, and pattern recognition.



Chao Zhang (Member, IEEE) received the Ph.D. degree in electrical engineering from Beijing Jiaotong University, Beijing, China, in 1995.

He is currently a Professor with the Department of Machine Intelligence, School of Electronics Engineering and Computer Science, Peking University, Beijing. His research interests include computer vision, image processing, and pattern recognition.

Authorized licensed use limited to: Pontificia Universidad Catolica Del Peru. Downloaded on October 28,2026 at 00:10:34 UTC from IEEE Xplore. Restrictions apply.